Aalto University
School of Science

Observability and Analytics Experimentation

Hong-Tri Nguyen
Department of Computer Science
hong-tri.nguyen@aalto.fi

CS-E4660 Advanced Topics in Software Systems, Fall 2026
September 9, 2026

Learning objectives

- Foundation of observability and analytic experimentation
- The usage/design of observability and analytic experiment framework for
  - Service-based software system
  - Hybrid Intelligence System (HIS)

Outline

From previous lectures

Service-based hybrid intelligence software systems

Current computing era

Enable:

- Service-based applications/systems
- Muti-continuum computing with diverse types

Service-based hybrid intelligence software systems

Current computing era

Enable:

- Service-based applications/systems
- Muti-continuum computing with diverse types

Hybrid Intelligence Systems (HIS)

Service-based hybrid intelligence software systems

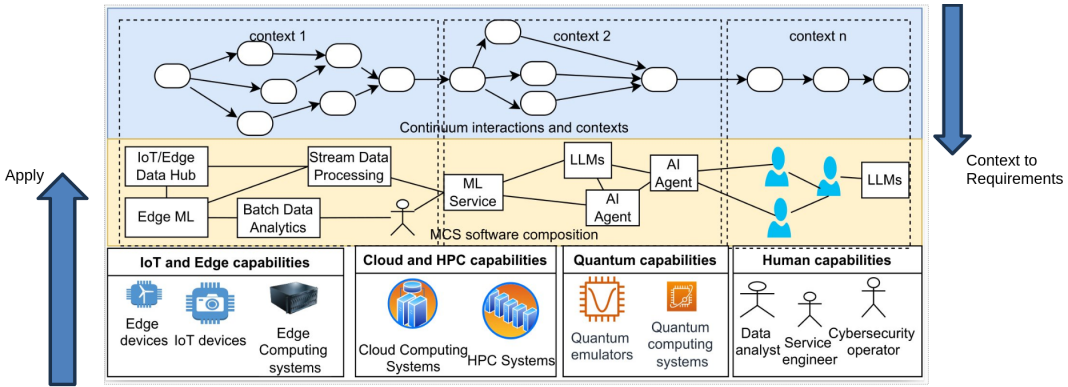


Figure: Muti-continuum service-based applications

Use-case object classification

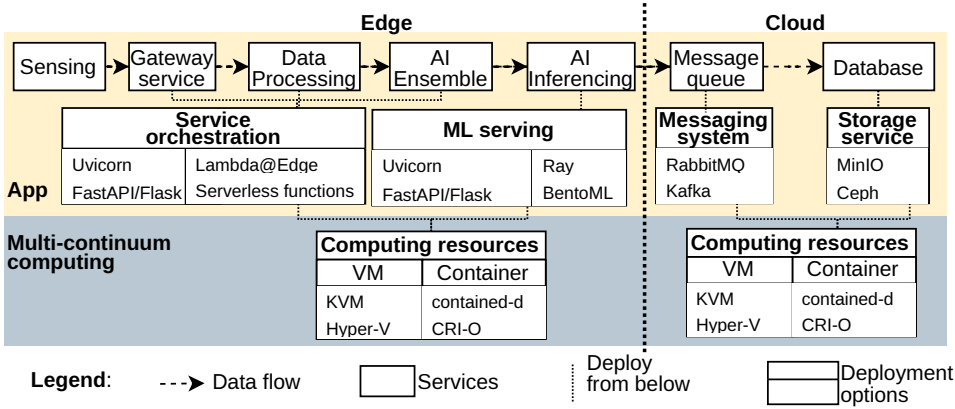


Figure: An edge-cloud ML-based application[1]

[1] https://github.com/rdsea/object_classification_v2

From benefits to challenges

Benefits from compositions of services and capabilities

+ Enable new applications
+ Optimize specific dimensions (performance, cost)
  - Performance
  - Cost
  - Scalability
+ Enable elasticity
  - Allow fine-grained scaling of components
  - Recover from failures

From benefits to challenges

Benefits from compositions of services and capabilities

+ Enable new applications
+ Optimize specific dimensions (performance, cost)
  - Performance
  - Cost
  - Scalability
+ Enable elasticity
  - Allow fine-grained scaling of components
  - Recover from failures

Challenges from compositions of services and capabilities

- Increase orchestration and configuration complexity
- Introduce additional latency
- Create new dependency-failure risks

From benefits to challenges

Composition of various services and capabilities therefore requires high-quality analytics for further decisions

From quality of analytics to quality of decisions

Quality of Analytics

Analytics should provide information for a system, which can be viewed:

- as a process
- as an artifact
- as a service

From quality of analytics to quality of decisions

How to assess Quality of Analytics?

We need evidence about:

- **Functions**: requirements and constraints
- **Behaviors**: performance, failures, scalability, and cost
- **Structure**: services, dependencies, deployment, and configuration

From quality of analytics to quality of decisions

How to assess Quality of Analytics?

We need evidence about:

- **Functions**: requirements and constraints
- **Behaviors**: performance, failures, scalability, and cost
- **Structure**: services, dependencies, deployment, and configuration

Techniques:

- **Observability**: evidence from the running system
- **Controlled experimentation**: evidence from understanding/comparing among alternatives decisions

Observability [1, 2]

From control theory to system/software engineering

Observability

"Observability" from Rudolf E. Kalman work [3] in 1960 as the birth of modern control theory, which indicates the dual exist between observability and controllability

- System is to measure and infer the complete internal state of the system
- Software means to interact and understand the code

Observability: the ability to understand/debug any given system state

Our purpose: reliability and performance

"Observability isn't just a debugging technique or a category of tooling; it's a property of software dependability, as fundamental as **reliability** or **availability**"

Monitoring vs. observability

Monitoring vs. observability

Monitoring works for

- Presents up/down/underperformed sites

Monitoring vs. observability

Monitoring works for

- Presents up/down/underperformed sites

Observability is for large-scale systems

- Monitoring is not enough for **why** with flexibly scale
- Observability tools to tell what happens to that request

Observability workflow: collecting and analyzing

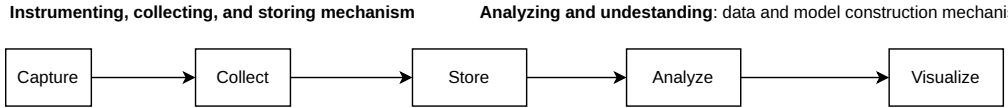


Figure: Observability workflow

Observability workflow: collecting and analyzing

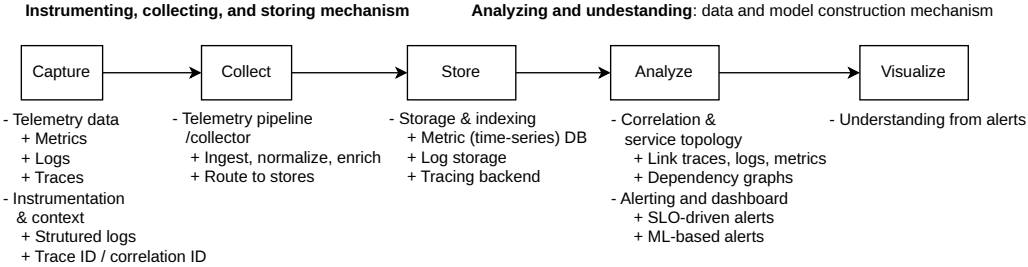


Figure: Observability workflow

Design observability: an example

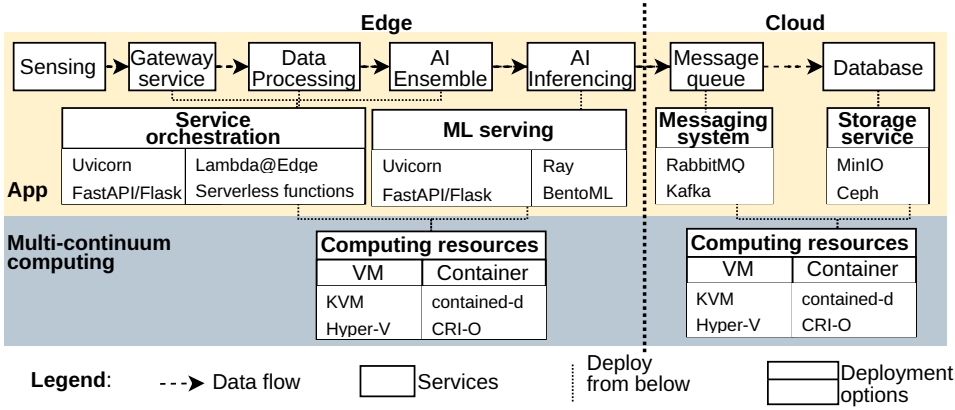


Figure: An edge-cloud ML-based application

Design observability: an example

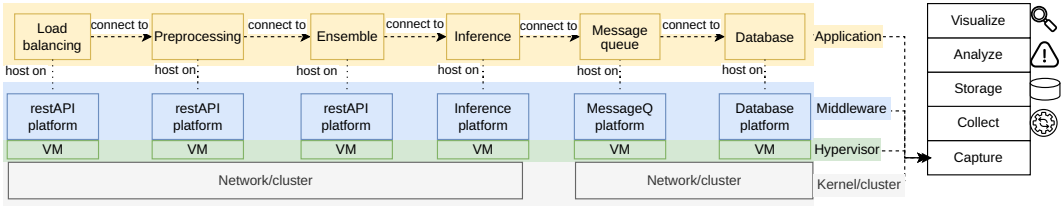


Figure: Layer-service architecture

Three pillars vs unified data

Three pillars model

- Each signal type gets streamed off and stored in its own
  - Metric to time-series database
  - Log to log-aggregation service
  - Trace to tracing service
- Most usage, because of effort, cost, infrastructure, and operations have owned the tooling, and the three pillars suit third-party needs

Unified data

- All telemetry describing a request is stored once to preserve:
  - Contexts
  - Relationships
- Challenges: different structures, difficult/complex query, different retention needs, and high volume/cost

Metric


```
{
"timestamp": "6:00:00",
"type": "count",
"name": "http.request.count",
"value": 212,
"server_id": "web.1",
"region": "north_america"
}
```


- Metric provides a broad view of system health, consisting of quantitative data that measures various aspects of system performance and resource utilization

Metric


```
{
"timestamp": "6:00:00",
"type": "count",
"name": "http.request.count",
"value": 212,
"server_id": "web.1",
"region": "north_america"
}
```


- Metric provides a broad view of system health, consisting of quantitative data that measures various aspects of system performance and resource utilization

Challenges

- Cannot provide a context
- Lack of relationships among metric data, sequential triggering of alerts

Log

Time	Message
6:01:00	Accepted connection on
6:01:03	Basic authentication accepted for user: `foo`
6:01:15	Processing request
6:01:18	Request succeeded
6:01:19	Closed connection to ...

- Log are detailed chronological records of specific events that occur within a system
- Log files are large blobs of unstructured text, designed to be readable by humans but difficult for machines to process

Log

Time	Message
6:01:00	Accepted connection on
6:01:03	Basic authentication accepted for user: `foo`
6:01:15	Processing request
6:01:18	Request succeeded
6:01:19	Closed connection to ...

- Log are detailed chronological records of specific events that occur within a system
- Log files are large blobs of unstructured text, designed to be readable by humans but difficult for machines to process

Challenges

- Unstructured logs require parers to split into chunks
- Challenges to unstructured nature, diverse formats, vast volume

Trace [4]

- Distributed tracing is a method to monitor applications, by recording and tracking or logging end-to-end requests when they flow through various services or components
- Trace tracks end-to-end insight the flow of a request as it travels through multiple components of a system
  - Each step in this sequence is called a span, and spans can contain their own detailed information about what occurred during that operation
- A way to represent an interrelated series of events

Challenges

- High data volume: one request can generate many spans
- Storage and query overhead for reconstructing traces

Trace [4]

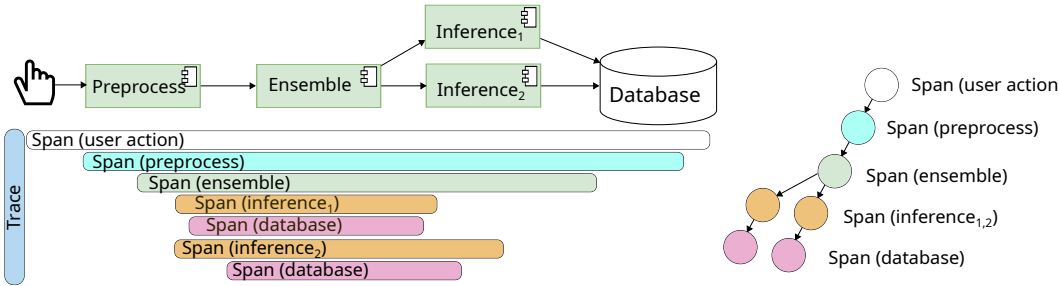


Figure: An example for a trace

Cost/Volume management/Processing

Challenges

- High telemetry volume increases storage, network, and processing costs
  - Facebook (Canopy [5]) reported 1.3 billion traces per day in 2017
  - Google (Dapper [6]) collected over 1 TB of sampled trace data per day in 2010
- Querying and correlating large datasets can be computationally expensive
- Disconnected telemetry increases the human cost of diagnosing incidents

Samplings

- Head sampling: decide whether to keep telemetry before or during collection
- Tail sampling: make the sampling decision after observing the telemetry

Compare telemetry data

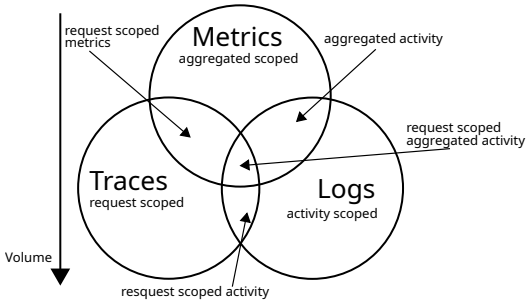


Figure: Comparison

- Metrics for precise fault localization via considering performance indicator at specific point (fast alert and inexpensive long-term storage)
- Log has a lot of operational status information, error/warning messages for forensic evidence a single process or transaction (audit requirement)
- Trace has valuable for documenting and analyzing request path to show critical insights interconnected relationships (causality and full-request context)

Architecture for collecting and storing telemetry data

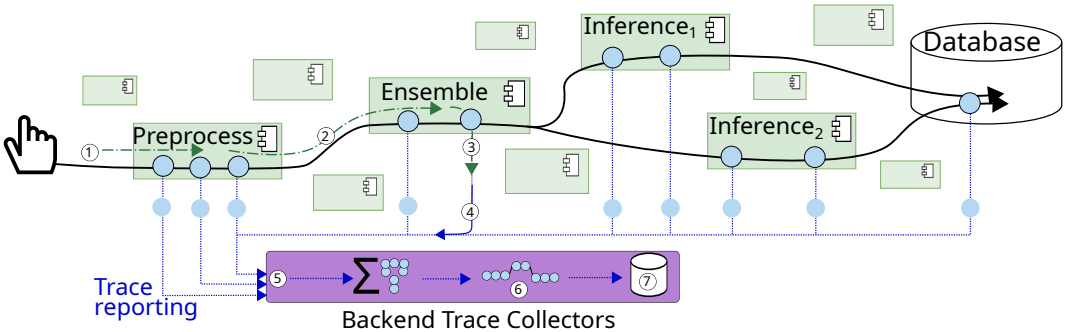


Figure: A request traverses system processes [7]: (1) assigning a unique traceID, (2) propagating traceID and sampled flag, (3) annotating with traceId, (4) transmitting trace data, (5) backend receives, (6) processing, (7) storing

Collecting techniques or instrumentation

Instrumentation

- Def: adding codes to applications to emit data
  - Remind: observability helps to answer questions
  - Understanding behaviors, performance, error diagnosis
- Analogy: installing sensors and meter throughout the applications

Types

- Manual instrumentation
- Auto-instrumentation

Instrumentation strategy is needed and evolves alongside with the applications

Manual instrumentation

Customize/Manual instrumentation works by capturing telemetry that is specific to the application's business logic

- Understand the specific state of things: user actions, business processes, and domain-specific operations

Manual instrumentation

Customize/Manual instrumentation works by capturing telemetry that is specific to the application's business logic

- Understand the specific state of things: user actions, business processes, and domain-specific operations

Challenges: Take time and effort

- Systems with various applications written in different languages
- Require source code modification

Auto instrumentation

Automatic type works by intercepting common operations in the application framework and libraries

- Breadth of coverage with minimal effort
- Types:
  - Instrumentation library: receives a call and produces telemetry data, then calls the underlying library

Auto instrumentation

Automatic type works by intercepting common operations in the application framework and libraries

- Breadth of coverage with minimal effort
- Types:
  - Instrumentation library: receives a call and produces telemetry data, then calls the underlying library

Challenges: Noise

- Hard to instrument application-specific code
- Instrument useless things

Our

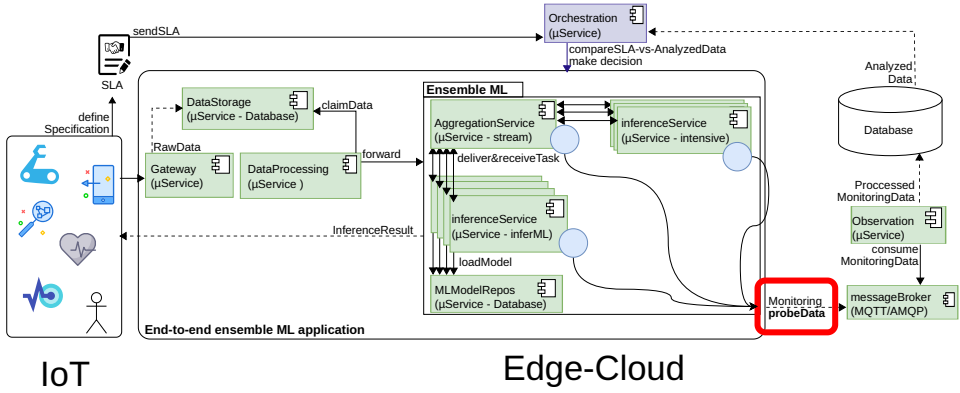


Figure: Quality of analytics for ML [8]

Limitations of current observability for HIS

1. AI components introduce non-deterministic behavior
   - Same input may produce different outputs
   - Output depends on prompt design, context, and model version

Limitations of current observability for HIS

1. AI components introduce non-deterministic behavior
   - Same input may produce different outputs
   - Output depends on prompt design, context, and model version
2. Understanding the system becomes more difficult
   - Failures may come from data, prompts, models, or human-AI interaction
   - Root-cause analysis requires observing both technical and intelligent behaviors

Limitations of current observability for HIS

1. AI components introduce non-deterministic behavior
   - Same input may produce different outputs
   - Output depends on prompt design, context, and model version
2. Understanding the system becomes more difficult
   - Failures may come from data, prompts, models, or human-AI interaction
   - Root-cause analysis requires observing both technical and intelligent behaviors
3. Need to rethink observability for HIS
   - Observability should include prompts, model versions, outputs, feedback, and decision context

Observation in LLM-based systems

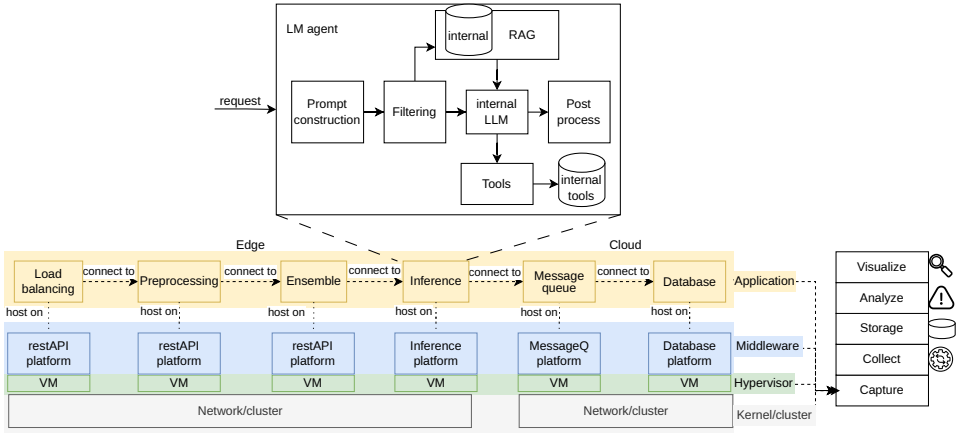


Figure: Observation for LLM-agent systems

Observability for AI agents

Why need observability for AI agents

When agents can write, test, review and ship code faster, verifying these codes becomes bottleneck

- Non-deteministic harder to root-cause
- New failure modes: hallucination, prompt injection, retrieval failure

What to be captured

- Prompt / model response along model metadata
- Similarity score of documents (RAG provenance)

Example: AI observability framework

AgentSight [9]

Single agent: AgentSight intercepts LLM traffic instead of source for extracting semantic intent (Gap between Intent and Action) and kernel events (dynamic inkernel eBPF filter)

LumiMAS [10]

Multi-agent system: LumiMAS works to detect hallucination and bias Monitoring: application start/end, Agent start/end,LLM calls, tool usage, token usage, semantic interaction for Root cause analysis with LLM-based agent

Observability tools for AI agent

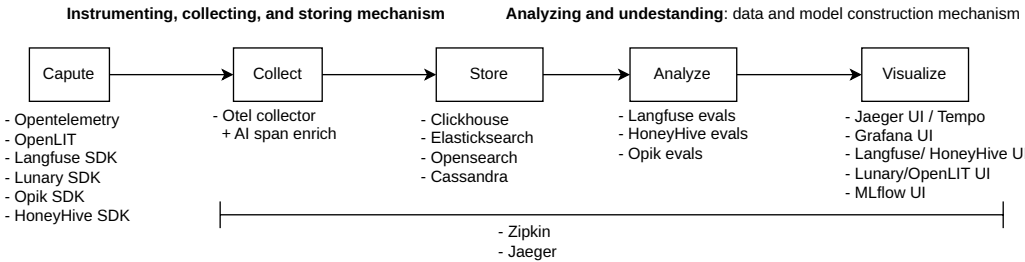


Figure: Observable tools for AI[2]

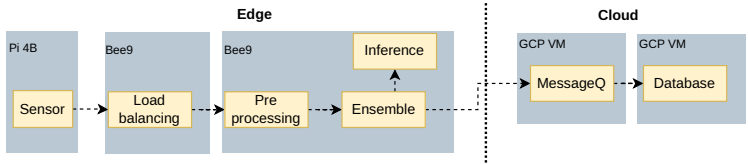
[2] https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.173473181.10918695/v1

Controlled experimentation framework [11]

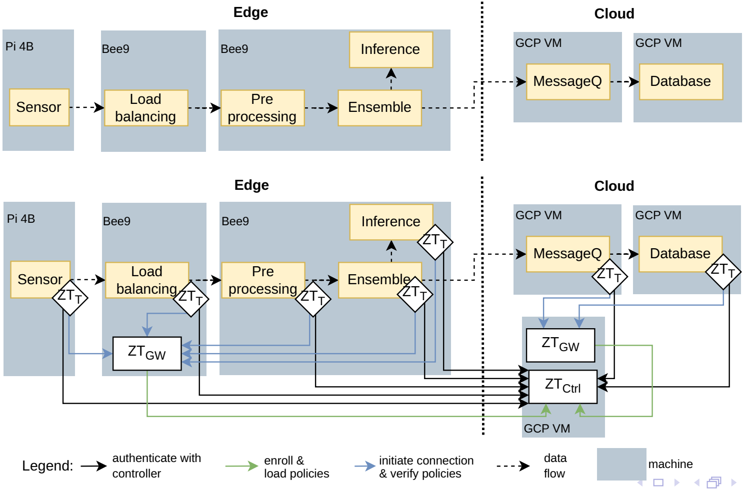
Motivation

- How to introduce a new feature and compare with the existing system?
- What is the best/ the most suitable composition of services and capabilities for service-based software systems?

Motivation


Edge
Cloud
Pi 4B
Bee9
Bee9
Inference
GCP VM
GCP VM
Sensor
Load balancing
Pre processing
Ensemble
MessageQ
Database

Motivation



Edge
Cloud
Pi 4B
Bee9
Bee9
GCP VM
GCP VM
Sensor
Load balancing
Pre processing
Ensemble
Inference
MessageQ
Database
Edge
Cloud
Pi 4B
Bee9
Bee9
GCP VM
GCP VM
Sensor
Load balancing
Pre processing
Ensemble
Inference
MessageQ
Database
ZT_T
ZT_GW
ZT_Ctrl
GCP VM
Legend:
authenticate with controller
enroll & load policies
initiate connection & verify policies
data flow
machine

Motivation

Is the added security worth the cost?

- What overhead does it introduce in latency, resource usage, and complexity?
- How can we evaluate this scientifically and reliably?

Motivation

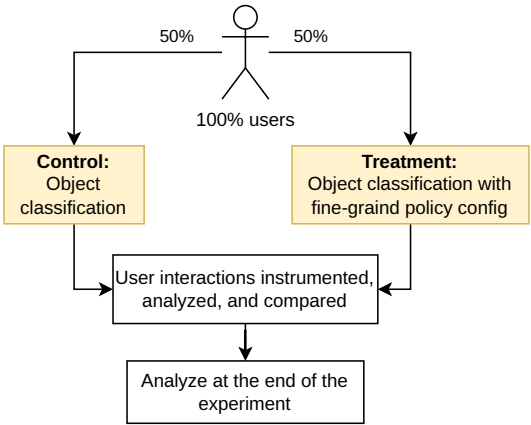


Figure: A controlled experiment

A simple controlled experiment

Want Assess the value of an idea

But Running an experiment is small overhead

Need A clear overall evaluation criterion

Usage Many usage from big tech [12]

Why do we need experiment platforms?

Controlled experiments

- Quantify the causal impact of architectural changes
- Compare the existing system with a modified system
- Measure both benefits and costs: latency, resource usage, reliability, and trustworthiness

Engineering challenge

- Each experiment requires deployment, configuration, workload generation, and monitoring
- Manual experimentation increases engineering effort and risk of mistakes
- Results must be reproducible to support reliable decision making

Why do we need experiment platforms?

Experiment platform

- Provides reusable infrastructure for running experiments
- Automates data collection and analysis
- Reduces engineering effort while improving reproducibility and trust

Controlled experiment terminology

- **Overall evaluation criterion (OEC)**: A quantitative measure of the experiment's objective
- **Parameters**: a controllable experimental variable to influence OEC
- **Variants**:
  - Control as the baseline/existing system
  - Treatment as the existing system along with features
- **Randomization units**: process is applied to units to map them to variants

Designing experiment

- What is the randomization unit?
  - The randomization unit is a user

Designing experiment

- What is the randomization unit?
  - The randomization unit is a user
- What population of randomization units do we want to target?
  - Target all users and analyze those who visit/operate specific tasks

Designing experiment

- What is the randomization unit?
  - The randomization unit is a user
- What population of randomization units do we want to target?
  - Target all users and analyze those who visit/operate specific tasks
- How large (size) does our experiment need to be?
  - Need an analysis to determine size

Designing experiment

- What is the randomization unit?
  - The randomization unit is a user
- What population of randomization units do we want to target?
  - Target all users and analyze those who visit/operate specific tasks
- How large (size) does our experiment need to be?
  - Need an analysis to determine size
- How long do we run the experiment?
  - Run the experiment for a full week to ensure the understand

Experimentation platform

High-level components [12]

- Experiment definition, setup, and management

Experimentation platform

High-level components [12]

- Experiment definition, setup, and management
- Experiment deployment (architecture)
  - An experimentation infrastructure that provides experiment definitions, variant assignments, and other information
  - Production code changes that implement variant behavior according to the experiment assignment

Experimentation platform

High-level components [12]

- Experiment definition, setup, and management
- Experiment deployment (architecture)
  - An experimentation infrastructure that provides experiment definitions, variant assignments, and other information
  - Production code changes that implement variant behavior according to the experiment assignment
- Experiment instrumentation
  - To reflect new feature behaviors, help perform analysis
  - Scaling, concurrent experiments

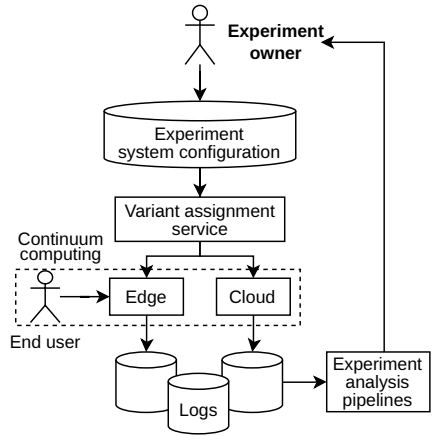
Experimentation platform

High-level components [12]

- Experiment definition, setup, and management
- Experiment deployment (architecture)
  - An experimentation infrastructure that provides experiment definitions, variant assignments, and other information
  - Production code changes that implement variant behavior according to the experiment assignment
- Experiment instrumentation
  - To reflect new feature behaviors, help perform analysis
  - Scaling, concurrent experiments
- Experiment analysis
  - choosing the goal, guardrail, and quality metrics is already done, as well as any codification of tradeoffs into an OEC

Experimentation platform architecture

1. Experiment specification
2. Experiment deploy and instrumentation
3. Experiment analysis and decisions



Our examples

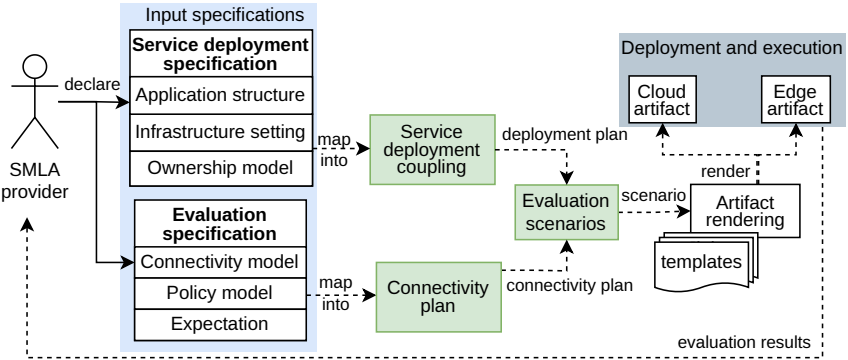


Figure: Our policy-configurations experiment framework

Our examples

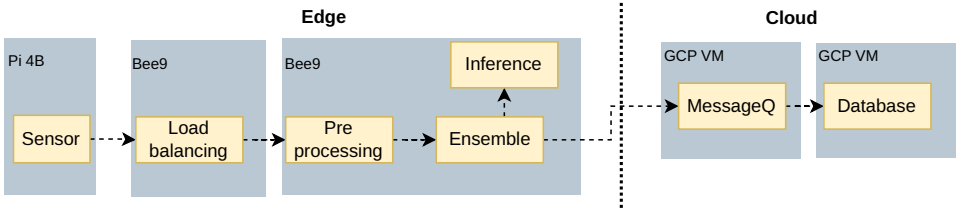


Figure: Control: baseline system

Our examples

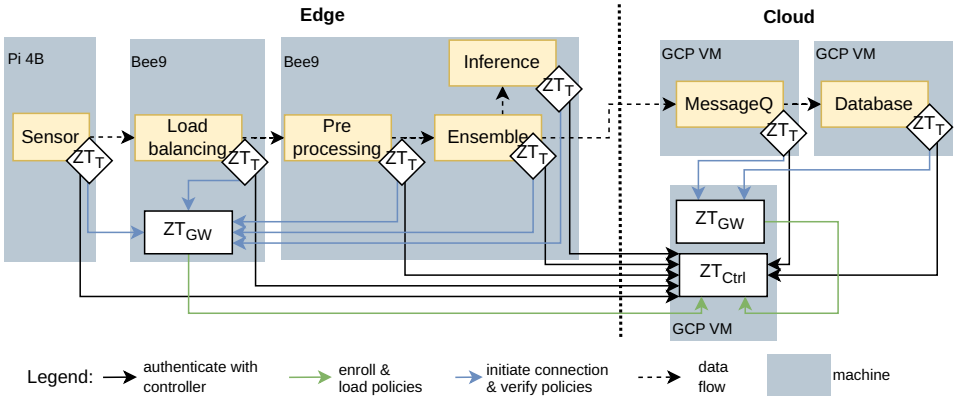
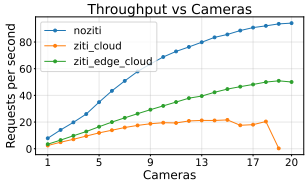
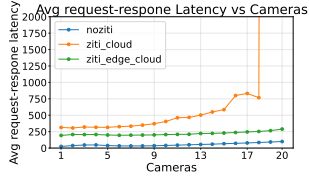


Figure: Treatment: baseline system with policy controller

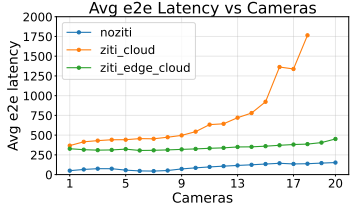
Our examples



(a) Requests per second vs cameras



(b) Average latency vs cameras



(c) Average end-to-end latency vs cameras

Our examples

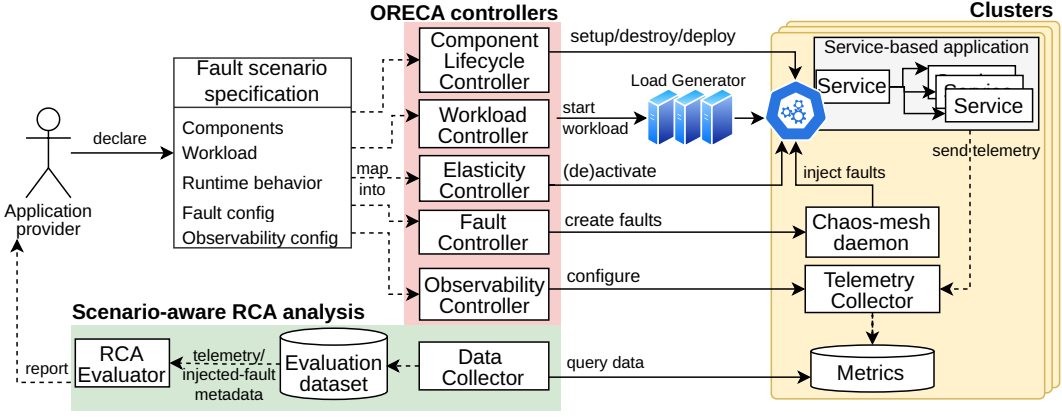


Figure: Our root-cause-analysis experiment framework

Our examples

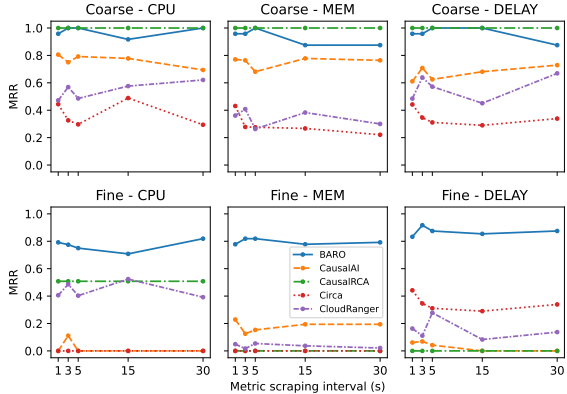


Figure: Metric sampling frequency effect on MRR

From experimentation platforms to HIS

- **Diversity of intelligence**
  - How can we experiment with systems that combine humans, LLMs, agents, rule-based components, and traditional software?
  - How can we evaluate trustworthiness not only of individual components, but of their interaction and composition?
- **Sovereignty and control**
  - How can we understand and control the workflow of LLM-based agents?
  - Where does data go during experimentation, and who has control over it?
- **Compliance and accountability**
  - How can experiments show that HIS behavior complies with regulations, policies, and organizational constraints?
  - How can we trace decisions back to requirements, data, prompts, models, tools, and human interventions?

From experimentation platforms to HIS

Measurement challenge

How can these concerns be operationalized as experimental metrics?

- **Diversity:** number/type of actors, autonomy levels, role distribution, interaction paths, decision dependencies.
- **Sovereignty:** data location, external API calls, token exposure, sensitive information leakage, workflow controllability.
- **Compliance:** requirement coverage, policy violations, audit logs, provenance completeness, explainability of decisions.

Take-home messages

- Observability: collecting and analyzing
- Data/instrumentation techniques have its pros/cons
  - Metric, log, trace
  - Auto/manual instrumentation
- Composition in multi-continuum computing extends experimentation to experimentation platform
- Design an experimentation platform
  - Define OEC, variants, randomization units, and parameters
- HIS, an open question for further work

Hands-on preparation

- Application: client/server, service-based application,
- Container: docker
- Cluster: minikube
- Instrumentation tool: opentelemetry
- LLM and tools: ollama, vllm with a simple model

Study logs

- What does it mean about the observability and analysis?
  - Trade-off or side effect from observation?
- Which components or steps in observation are the most important from your perspective? and why?
- Write short your thought on that and send to MyCourse using at least 1-2 references

Thank you

hong-tri.nguyen@aalto.fi

References I

[1] A. Boten, *Cloud-Native Observability with OpenTelemetry: Learn to gain visibility into systems by combining tracing, metrics, and logging with OpenTelemetry*. Packt Publishing Ltd, 2022.

[2] C. Majors, L. Fong-Jones, and G. Miranda, *Observability engineering: achieving production excellence*. " O'Reilly Media, Inc.", 2026.

[3] R. E. Kalman *et al.*, "On the general theory of control systems," *IFAC Proceedings Volumes*, vol. 1, no. 1, pp. 491–502, 1960.

[4] Y. Shkuro, *Mastering Distributed Tracing: Analyzing performance in microservices and complex systems*. Packt Publishing Ltd, 2019.

References II

[5] J. Kaldor, J. Mace, M. Bejda, E. Gao, W. Kuropatwa, J. O'Neill, K. W. Ong, B. Schaller, P. Shan, B. Viscomi, V. Venkataraman, K. Veeraraghavan, and Y. J. Song, "Canopy: An end-to-end performance tracing and analysis system," in *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, (New York, NY, USA), p. 34–50, Association for Computing Machinery, 2017.

[6] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure," 2010.

[7] L. Zhang, Z. Xie, V. Anand, Y. Vigfusson, and J. Mace, "The benefit of hindsight: Tracing Edge-Cases in distributed systems," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, (Boston, MA), pp. 321–339, USENIX Association, Apr. 2023.

References III

[8] L. Truong and T. Nguyen, "Qoa4ml–a framework for supporting contracts in machine learning services," in *IEEE International Conference on Web Services*, pp. 465–475, IEEE, 2021.

[9] Y. Zheng, Y. Hu, T. Yu, and A. Quinn, "Agentsight: System-level observability for ai agents using ebpf," in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*, PACMI '25, (New York, NY, USA), p. 110–115, Association for Computing Machinery, 2025.

References IV

[10] R. Solomon, Y. Yerushalmi Levi, L. Waknin, E. Aizikovich, A. Baras, E. Ohana, A. Giloni, S. Bose, C. Picardi, Y. Elovici, and A. Shabtai, "Lumimas: A comprehensive framework for real-time monitoring and enhanced observability in multi-agent systems," in *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*, AAMAS '26, (Richland, SC), p. 3749–3757, International Foundation for Autonomous Agents and Multiagent Systems, 2026.

[11] R. Kohavi, D. Tang, and Y. Xu, *Trustworthy online controlled experiments: A practical guide to a/b testing*.
Cambridge University Press, 2020.

References V

[12] S. Gupta, R. Kohavi, D. Tang, Y. Xu, R. Andersen, E. Bakshy, N. Cardin, S. Chandran, N. Chen, D. Coey, *et al.*, "Top challenges from the first practical online controlled experiments summit," *ACM SIGKDD Explorations Newsletter*, vol. 21, no. 1, pp. 20–35, 2019.